Group Learning Journey

IT3091 Machine Learning - Group 2026-AI-05

DataCo Smart Supply Chain: late-delivery risk

Bolonne B.R.M. (Rosheni), IT24102050 - exploratory analysis and preprocessing
Kodagoda D.S. (Sawanya), IT24101613 - exploratory analysis and baseline model
Jayasekara C.D. (Chamara), IT24103494 - exploratory analysis and hyperparameter tuning
Liyanage J.L.K.L. (Kavisha), IT24101927 - exploratory analysis, evaluation and recommendation

How we worked

We split the project along the pipeline rather than by task type, so each of us owned a stage from end to end and had to be able to explain it. Rosheni took the exploratory analysis and preprocessing, Sawanya the baseline model, Chamara the alternative models and tuning, and Kavisha the evaluation and the recommendation. All four of us worked on the exploratory analysis, because we wanted everyone to have seen the data before making decisions about it.

The handover points were the places where things could have gone wrong, so we made them explicit. The preprocessing stage wrote its output to files with a manifest describing every decision, and the modelling notebook loaded those files rather than repeating the work. That meant a change in one stage could not silently break another.

How we made decisions

We kept a decision log as we went rather than writing one at the end. It has seventeen entries, each with the option we chose, the alternative we considered, our reason and where the evidence sits in the notebooks.

The decisions we argued about most were these. Whether to use a random or a chronological split, which we settled by realising that one order spans up to five rows sharing a single label. Whether to apply PCA, which we settled by testing it and finding that feature selection beat it by 0.020 ROC-AUC at the same number of inputs. And whether to name a best model, which we decided against once we saw that the gap between the top three was smaller than the variation within any one of them.

What went well

Proving things instead of assuming them became our habit early, and it paid off. We suspected the target was derived from two other columns, so we tested it and found the rule reproduces the label on 97.55% of rows. That one check shaped the whole project, because it told us which columns had to go and why our scores would be modest rather than near perfect.

Keeping the exploratory analysis to the training set only was uncomfortable at first but turned out to be the right discipline. It meant none of our decisions were influenced by data we were supposed to be tested on.

We are also pleased that our main finding came from the data rather than the model. First Class is late 100% of the time because it promises one day and always takes two, and Second Class and Standard Class have identical actual transit times but very different late rates. That is a finding DataCo can act on without deploying anything.

What went wrong and what we redid

Our cross-validation leaked, slightly. We used a time-series split, which cuts at a row position, and because the rows of one order sit next to each other a fold boundary could land inside an order. We measured it and found two orders affected. It was a small number, but it was the same kind of leak we had rejected random splitting for, so we wrote a splitter that removes those rows and verified it reports zero.

We set the wrong expectation about the models. We assumed the tree models would barely beat Logistic Regression, and the first Decision Tree came out about two points better, so we corrected it before it reached the report.

We also nearly shipped a bad decision threshold. Optimising F1, which is the obvious thing to do, picked a cut-off that flags 97% of all orders. That is the always predict late model wearing a disguise, and it has a respectable looking score. We only caught it because we looked at how many orders were being flagged rather than at the metric alone.

Our workflow was not a straight line because of these. Findings in the exploratory analysis changed what we dropped in preprocessing, and the leakage check during evaluation sent us back to rewrite the cross-validation splitter.

How we used AI as a group

We agreed at the start that we would use an AI assistant openly and declare it, rather than use it quietly. We used it for writing and debugging code, for suggesting checks we had not thought of, and for drafting the report from results our notebooks had already produced.

We also agreed that nothing would go into the report unless one of us had run it and read the output. That rule earned its place. The assistant was confidently wrong several times, including recommending a cross-validation method that would not work with our data ordering, describing the encoder's handling of unseen categories incorrectly, and predicting that our test score would fall when it rose. Our report lists eight such errors with the corrections.

What we learned collectively is that the dangerous AI output is not the obviously wrong kind. It is the plausible kind that only fails when you check it against the data.

What we would do differently

We would run the leakage audit before any exploratory analysis rather than alongside it. Several of our early charts used columns we later had to drop.

We would decide the evaluation metrics and the business cost of each error type before training anything. We arrived at recall mattering more than precision quite late, and that decision should have shaped the work from the beginning.

We would also agree our file and handover conventions on day one. We lost time to confusion about which copy of a notebook was current, particularly when two of us had the same file open at once.

What we are taking away

The result we are most confident about is not the 0.7807 test score. It is that four one-hot columns describing the shipping mode reach 0.7387 on their own, so all of our modelling added about 0.03 over a rule that fits in a spreadsheet.

Being able to say that plainly, rather than presenting the model as more impressive than it is, feels like the real lesson of the assignment. The analysis told DataCo something useful about its delivery promises, and the model was the evidence for it rather than the product.